

CH5 Trees

General Trees

Tree is an abstract model of a hierarchical (分層) structure

A tree T consists of nodes with a parent-child relation

If T is nonempty, Root node which has no parent

Definition:

Siblings Nodes having the same parent

External Node (leaf) having no children is

Internal Node having child(ren) is

Ancestor Ancestor of E is E , B , A

Descendant Descendant of B is B , E , F , I , J , K

Subtree F is a subtree of B

Degrees The number of subtree a node directly connected to

Example: $\deg(A)=3$, $\deg(C)=2$

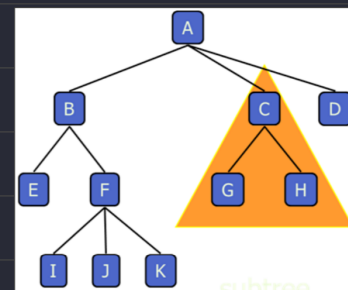
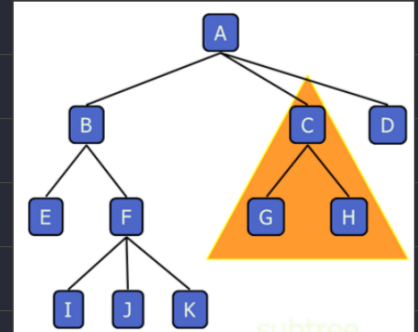
Edges A pair of nodes (u, v) such that u is the parent of v

Example: (A, B) , (B, F)

Path A sequence of nodes that any two consecutive nodes in it is an edge

Example: (A, B, F, J)

Leaf A node with no children



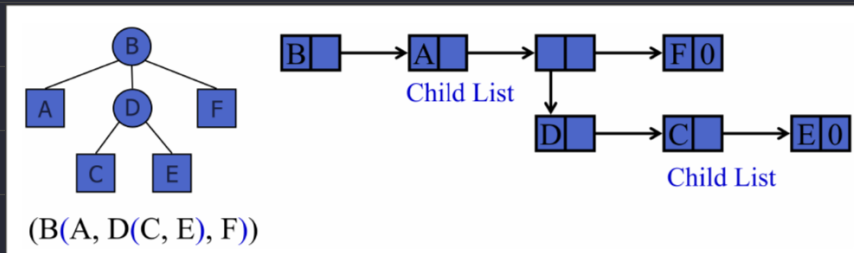
Ordered Trees

Tree T is ordered tree if the node in it can be listed in a specific way.

Typically listing the siblings in a correct order

Representation of Trees

Lists Representation



Binary Trees

Definition:

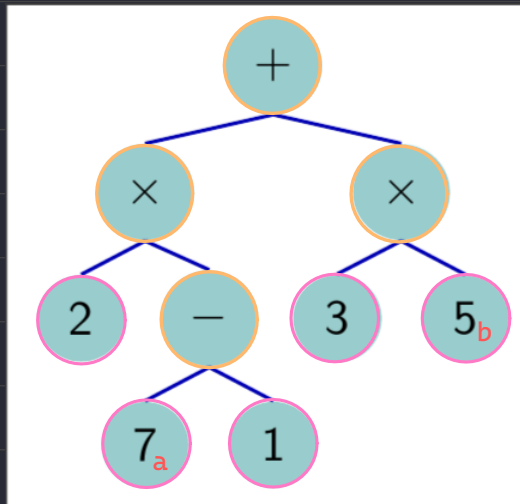
An **ordered** tree with two children: left, right

Left child precedes the right child in ordering

Left and right child are called subtree of it's root

Proper (full) binary tree: A tree that each node had either 0 or 2 children

Decision Tree



Node Type:

Internal nodes

External nodes

Properties:

[0] A full binary tree

[0] Have precedence: $(2 \times (a - 1) + (3 \times b))$

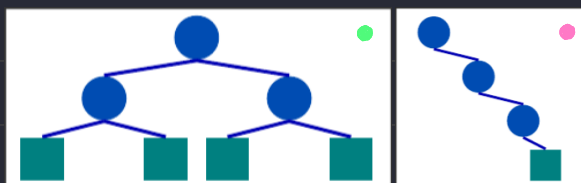
Properties of Binary Trees

Suppose T is a binary tree and

- n : number of nodes
- e : number of external nodes
- i : number of internal nodes
- h : height of T

Then,

- $h + 1 \leq n \leq 2^{h+1} - 1$ $2^0 + 2^1 + \dots + 2^h$
- $1 \leq e \leq 2^h$
- $h \leq i \leq 2^h - 1$ $2^0 + 2^1 + \dots + 2^{h-1}$
- $\log(n + 1) - 1 \leq h \leq n - 1$



Properties of Proper Binary Trees

Suppose T is a proper binary tree and Then,

- n : number of nodes
- e : number of external nodes
- i : number of internal nodes
- h : height of T

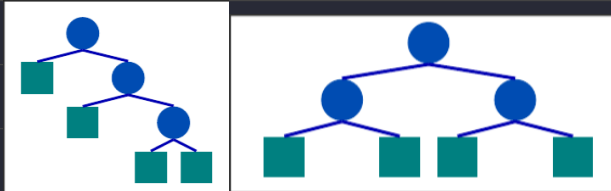
$$2h + 1 \leq n \leq 2^{h+1} - 1$$

$$h + 1 \leq e \leq 2^h$$

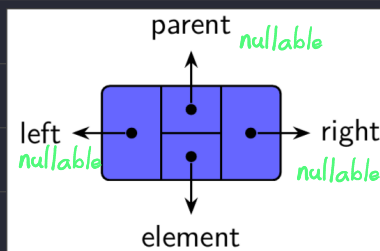
$$h < i < 2^h - 1$$

$$\log(n + 1) \leq h \leq \frac{n - 1}{2}$$

$$e = i + 1$$



Linked Structure for Binary Trees

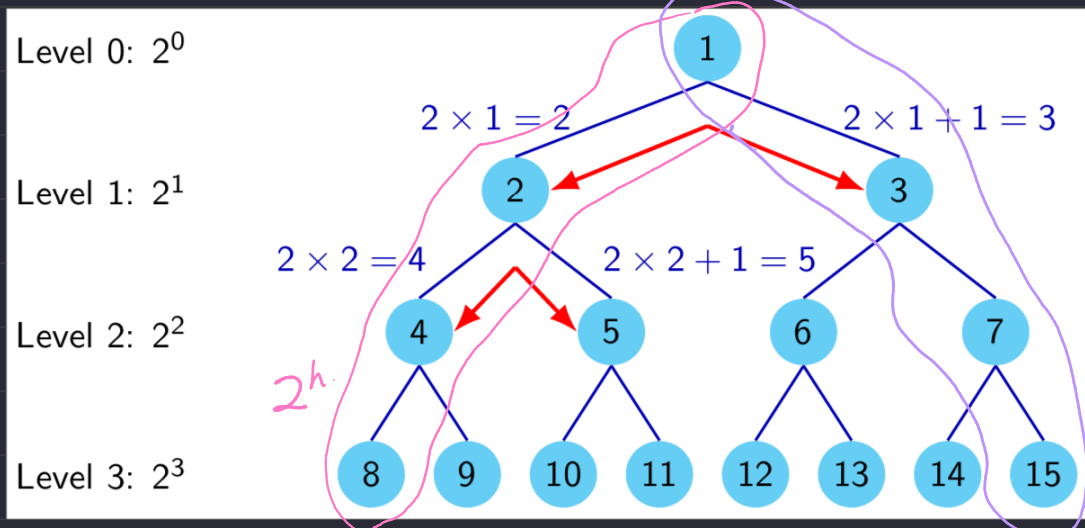


Array-based Structure

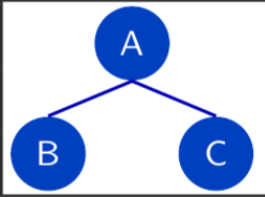
For every node v in T , define $p(v)$ as

- $p(v) = 1$, if v is the root
- $p(v) = 2p(u)$, if v is the left child of node u
- $p(v) = 2p(u) + 1$, if v is the right child of node u

The numbering function p is known as a level numbering of the nodes in T .



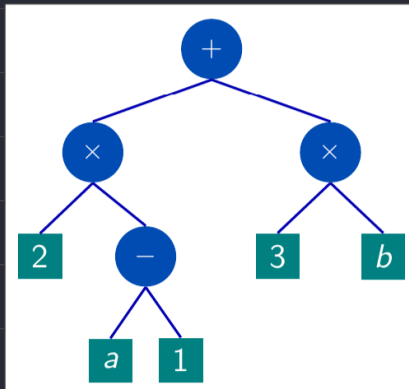
Traversals of a Binary Tree



Pre-order	Post-order	In-order
ABC	BCA	BAC

* General tree don't have in order, cuz the number of children is not fixed, which leads to uncertain middle place.

Print Arithmetic Expressions



PRINTEXPRESSION(v)

```

1  if v.hasLeft()
2      print("(")
3      PRINTEXPRESSION(v.left())
4  print(v.element())
5  if v.hasRight()
6      PRINTEXPRESSION(v.right())
7      print(")")
  
```

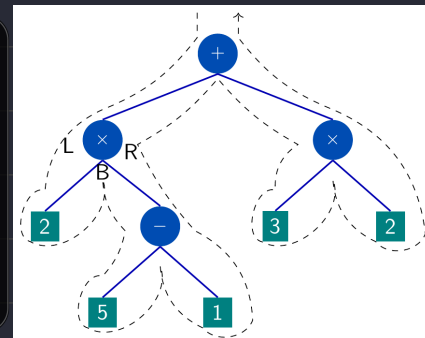
$((2 \times (a - 1)) + (3 \times b))$

Euler Tour Traversal

在每次的尋訪開始與結束時都作一個操作 e.g. 印出左右括號

```

1  def eulerTour(v)
2      # Perform the action for visiting node v on the left
3      if v.hasLeft()
4          eulerTour(v.left())
5      # Perform the action for visiting node v from below
6      if v.hasRight()
7          eulerTour(v.right())
8      # Perform the action for visiting node v on the right
  
```

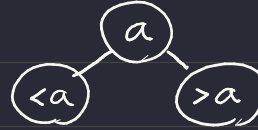


Binary Search Trees (BST)

Definition

Left node is smaller than the root

Right node is bigger than the root



Searching $\rightarrow O(h)$

```
if (key == item)
    return node
else if (key < item)
    return search(node.right)
else
    return search(node.left)
```

Insertion

```
def insert(root, value):
    """
    在 BST 中插入一個新值。
    :param root: 樹的根節點
    :param value: 要插入的值
    :return: 插入後樹的根節點
    """
    # 基本情況：如果當前節點是空的，則在此處建立新節點並回傳
    if root is None:
        return TreeNode(value)

    # 遞迴步驟：根據值的大小決定往左或往右走
    if value < root.val:
        root.left = insert(root.left, value)
    elif value > root.val:
        root.right = insert(root.right, value)

    # 如果 value == root.val, 我們什麼都不做，直接回傳原節點
    # 這樣可以避免插入重複的值

    return root
```

Deletion

```
if v is a leaf, then delete v directly
if v is an internal node having only one child u,
    then replace v by u and delete original u
if v is an internal node having two children,
    then select one subtree, say the right subtree;
    find the element y whose key is next to v;
    replace v by y and delete y
```

→ 在兩個節點的情況找右邊那策最小的節點交換，然後刪除 Leaf